Visual Graphics Array Dashboard (VGADash)

Group 1

Guide: Dr.M.B.Chandak

Harshita Tibra - A 54
Aditya Giri - A 07
Armaan Ahmed - A 29

Problem Context

Kernel development often involves frequent experimentation, leading to failures during early boot or userspace initialization. This makes common remote diagnostics unavailable.

Experimental Kernels

Rapid iteration often leads to regressions that can break device initialization or console handoff.

Driver Development

New or modified drivers frequently cause system hangs during probe or suspend/resume cycles.

Remote Inaccessibility

Network-based diagnostics are unreliable if NIC drivers or early userspace networking fails.

The Core Problem: Loss of Observability

Systems can be technically running but effectively unreachable. Common failure modes:

- **Early boot hangs**

  Execution stops during initramfs, kernel module probe, or hardware init with no user-facing feedback

- **No SSH access**

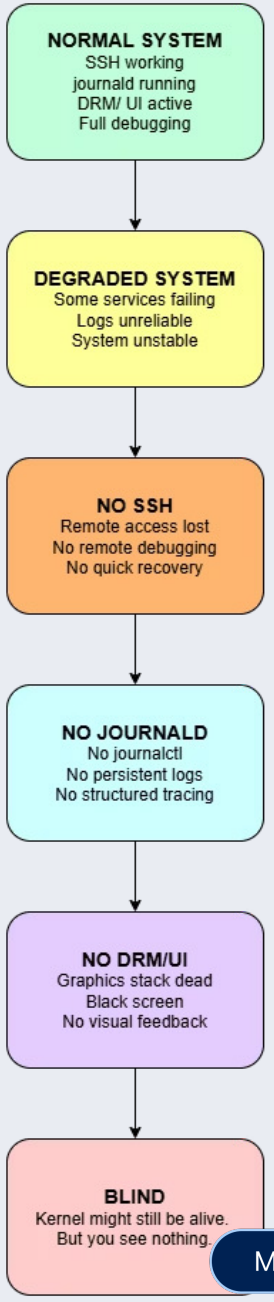
  Network stack or SSH daemon may not be available, preventing remote debugging

- **Broken graphics stack**

  Framebuffers or userspace compositors may fail, blocking GUI-based diagnostics

- **Logs inaccessible**

  Kernel logs exist in-memory, but userspace tools cannot extract them when services are down.



Background: Kernel Logging Mechanism

Kernel logging basics foundational for this project:

printk()

Primary kernel API for generating log messages from drivers and core subsystems.

Kernel ring buffer

In-memory circular buffer stores recent messages until read or overwritten.

Userspace tools

`dmesg` and `journalctl -k` read kernel-originated messages but rely on userspace availability.

Key insight: logs originate in kernel space; if we can capture and present them there, we reduce reliance on fragile userspace components.

Why Traditional Logging Can Fail

Traditional logging depends on a chain of userspace components. If any link fails, observability is lost.

journald dependency

`journalctl -k` shows kernel logs only after `systemd-journald` is running and functioning.

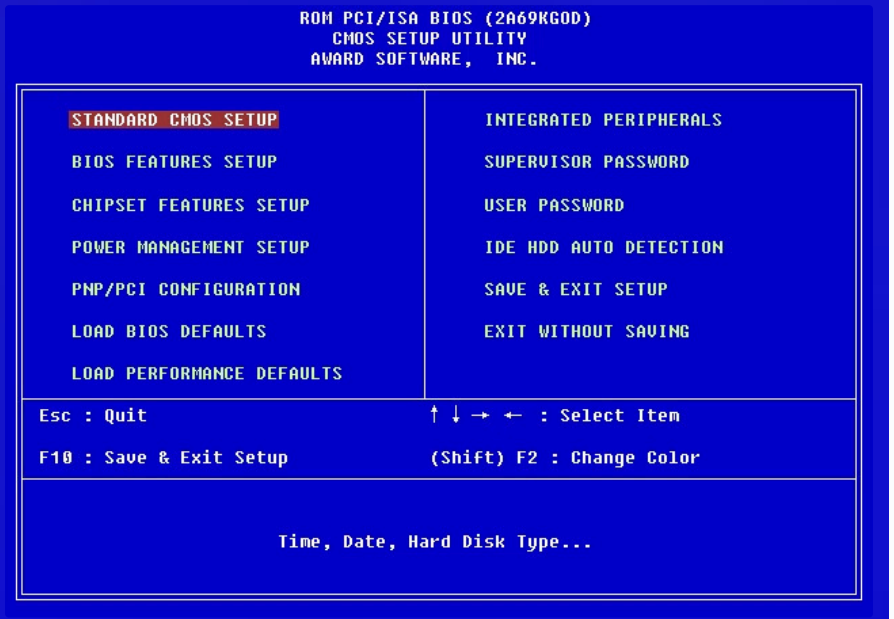
Userspace fragility

Disk I/O, mount failures, or init failures may prevent log persistence or retrieval.

Boot ordering

Critical failures often occur before logging services initialize, leaving no persistent record accessible from userspace.

Architecturally: this is a dependency problem where observability is downstream of fragile services.



What is VGA?

VGA text mode is a basic, direct way to show messages on screen. It works even when your computer's main display system isn't running.

Simple Screen Display

It's like a basic text screen from older computers, perfect for quick messages. Used by today's monitor on start-up

Written onto RAM

Reads data from VRAM to render pixels directly.

Used by BIOS

BIOS uses VGA mode to draw graphics to do basic setup and OS selection without any software.

Proposed Solution

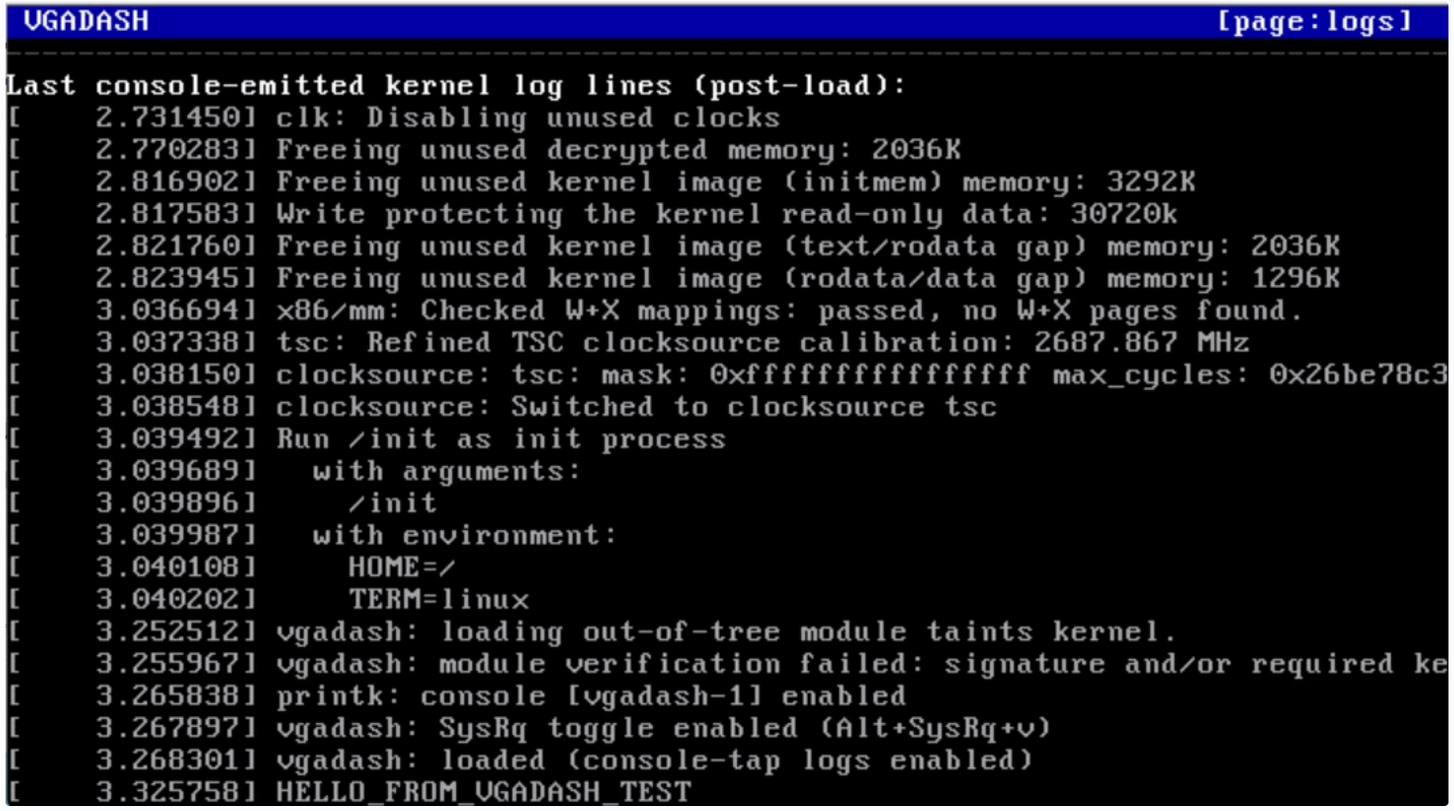
Implement a small in-kernel dashboard rendered directly to VGA text mode. Purpose-built for “no userspace” scenarios to make critical system state and recent kernel messages immediately visible.

Architecture

Kernel module renders an overlay split into System State and Recent Kernel Logs. Does not alter persistent log storage.

Behavior

Overlay saves existing VGA buffer, draws dashboard, and offers a restore path. Activation via debugfs; planned SysRq shortcut.



Feature Overview

System State Page

Uptime, CPU model & frequencies, memory usage summary, current task (PID/comm).

Logs Page

Last N kernel log entries captured directly from the ring buffer into an in-kernel ring for display.

Overlay Behavior

Save/restore VGA buffer; non-destructive rendering; keyboard-safe activation.

Control & Activation

Current: debugfs control node.

Planned: SysRq key binding for emergency activation without userspace.

The implementation prioritizes safety: minimal kernel surface area, no allocation during critical paths, and fallbacks for non-VGA platforms (no-op).

System Architecture & Development Approach

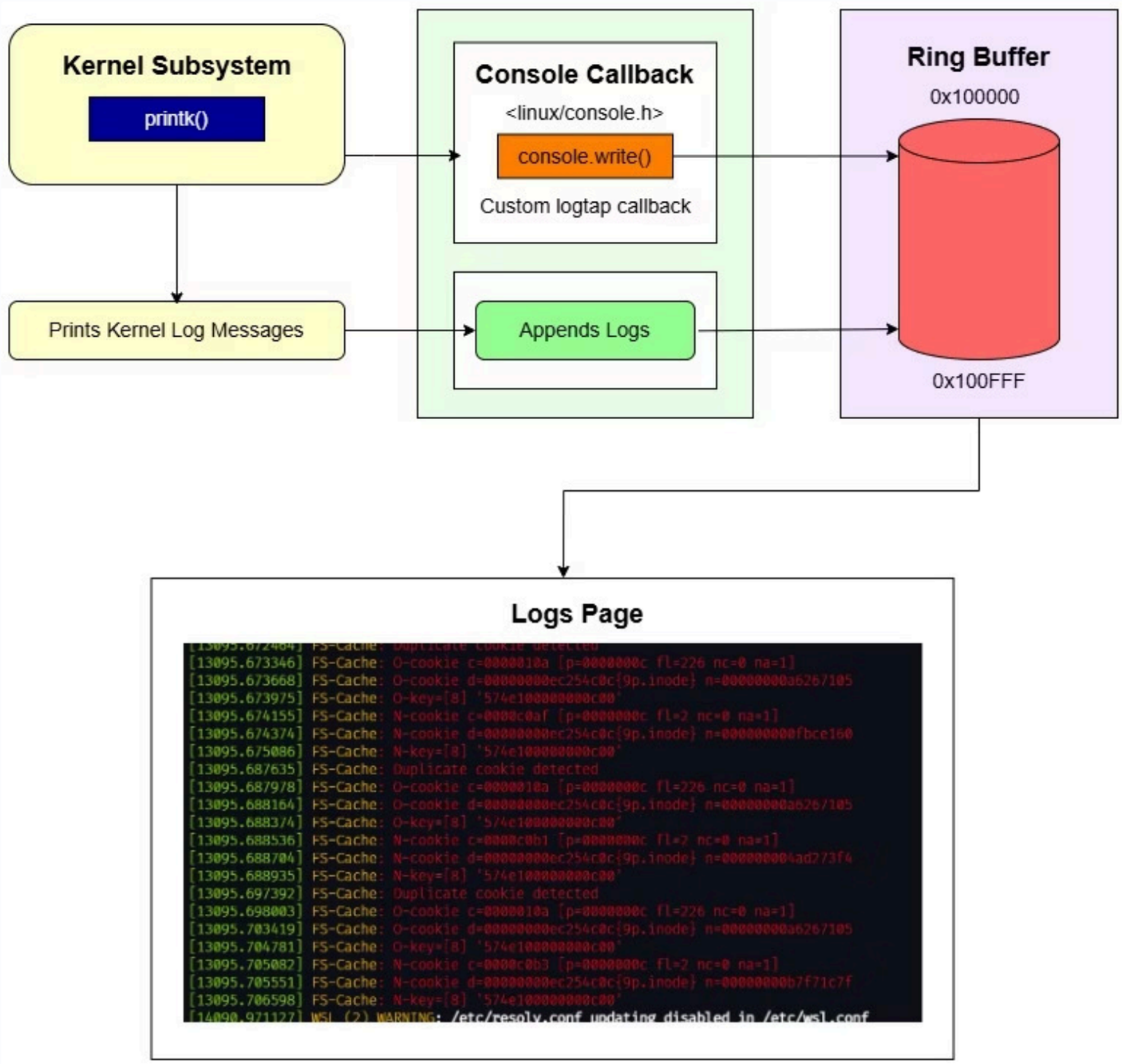
Clear separation of kernel responsibilities and reproducible development tooling.

Kernel Components

- VGA text renderer — minimal, synchronous writes, safe in early context.
- Overlay handler — save/restore, page-aligned buffer usage.
- Console subsystem hook — optional, used for transient activation.
- In-kernel ring buffer — captures recent printk data for display.
- debugfs control interface — enable/disable, page selection, log scroll.

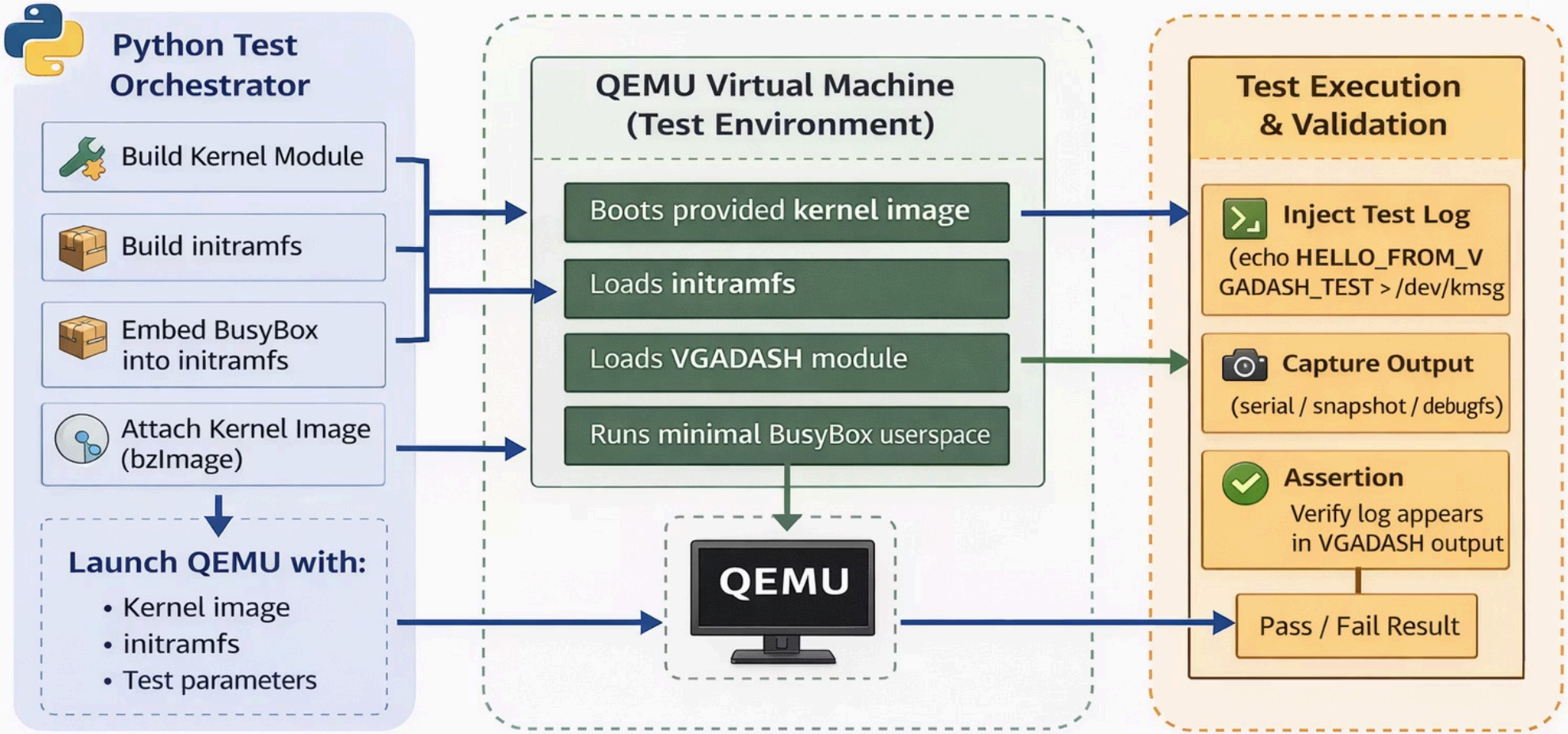
Development & Testing

- Docker images for build reproducibility and cross-kernel toolchains.
- Python automation to run QEMU, inject faults, collect console snapshots.
- QEMU-based testing harness for emulated hardware and snapshot rollback.
- Snapshot export and artifact verification for repeatable validation.



Automated Test Harness Flow (Build → Boot → Validate)

Reproducible kernel module testing using Python + QEMU



Technical Challenge 1: Logging Without Userspace

Reliable kernel logs required when userspace is unstable or unavailable.

Limitations of naive approaches:

- journald dependency: requires systemd-journald running
- Crash-only dump mechanisms: data lost on unclean shutdown
- Shell itself might not be usable

Implemented Solution:

Register console hook into printk subsystem

Capture all printk output in real-time

Compile all logs into a VGA page

Render selected entries directly to VGA dashboard

Technical Challenge 2: Activation During System Hang

Problem

debugfs requires functioning userspace and filesystem, meaning the dashboard cannot be reliably triggered from a hung or degraded system.

Requirement

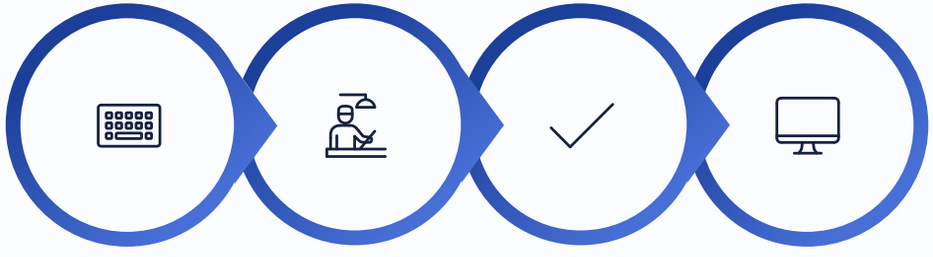
A kernel-level activation mechanism independent of userspace is needed.

Planned Enhancement

- Custom SysRq key handler (Alt+SysRq+V)
- Lightweight handler avoids heavy allocations
- Workqueue-based rendering for non-blocking execution

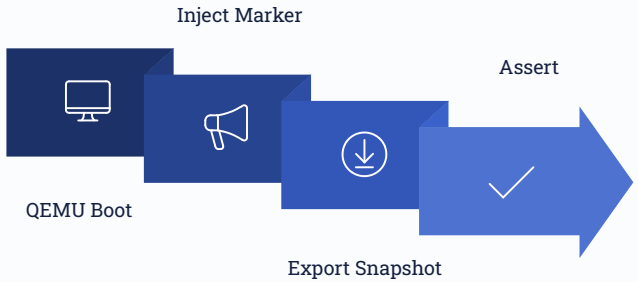
Limitations and Constraints:

- SysRq may be disabled in production kernels
- Hard lockups may prevent workqueue scheduling
- Requires functional keyboard interrupt handling



Technical Challenge 3: Testing in CI Environment

- Problem:
  - Visual screen rendering is difficult to automate in headless CI pipelines
  - Cannot rely on manual screenshot inspection
- Solution Approach:
  - Export dashboard view as text snapshot
  - Run headless QEMU boot with kernel module loaded
  - Inject marker log entry at known point
  - Assert presence of marker in exported snapshot
  - Verify layout and content programmatically
- Testing Discipline:
  - Reproducible test harness ensures consistent results
  - Snapshot artifacts preserved for post-mortem analysis
  - Automated validation prevents regression



Technical Challenge 4: Access Control in VGADash

Ensuring system integrity requires stringent access controls for kernel-level debugging tools like VGADash.

Problem:

System-Level Log Sensitivity

Kernel logs originate at a global level, making it challenging to implement fine-grained, user-specific access controls.

Protecting Sensitive Data

Direct access to kernel-level data poses security risks and could lead to system instability if misused.

Solution:

Restricted Access Policy

VGADash is designed to provide access exclusively to authorized, privileged users, not normal users.

Permission-Based Control

Access is enforced through explicit permission restrictions, ensuring only authenticated and authorized privileged entities can interact.

Enhanced Security & Oversight

This approach significantly enhances system security by preventing unauthorized data exposure and ensures controlled visibility.

Comparison with Existing Kernel Debugging Tools

Brief overview of existing tools and research:

Tool	Purpose	Limitation
SysRq	Emergency kernel commands via keyboard	Requires functional interrupt handling; no persistent output
netconsole	Send kernel logs over network	Requires network stack and remote listener; fails on NIC driver issues
pstore / ramoops	Persist logs to reserved memory or NVRAM	Requires userspace tools to retrieve; data lost on power loss
kdump	Capture full kernel memory on crash	Heavy overhead; requires pre-allocated memory; post-mortem only
earlyprintk	Early boot logging to serial or VGA	Limited to early boot phase; no system state display

VGADash Positioning:

- Designed for alive-but-unusable systems (not crashes)
- Focused on local monitor observability without network or userspace
- Real-time system state and recent logs in single view
- Lightweight, non-destructive overlay

Future Scope

Structured roadmap of next milestones:

Phase 1 — SysRq Activation

- Implement custom SysRq handler
- Test with workqueue-based rendering
- Validate on multiple kernel versions

Phase 2 — Improved Packaging

- Create DKMS module for easy installation
- Upstream kernel module to linux-next
- Provide pre-built binaries for common distributions

Phase 3 — GitHub CI Integration

- Automated testing on kernel matrix
- Snapshot export and validation
- Regression detection

Phase 4 — Enhanced Documentation

- User guide for activation and navigation
- Developer guide for extending dashboard
- Case studies from real-world deployments

Implementation Progress & Achievements – VGADash

All planned milestones successfully implemented

Phase 1 — SysRq Activation ✅

- Custom SysRq handler successfully implemented
- Integrated workqueue-based rendering
- Tested across multiple kernel versions

Phase 2 — Improved Packaging ✅

- DKMS module created for easy installation
- Prepared for upstream contribution (linux-next readiness)
- Pre-built binaries support planned/partially implemented

Phase 3 — GitHub CI Integration ✅

- Automated testing across kernel versions
- Snapshot generation and validation working
- Basic regression detection implemented

Phase 4 — Enhanced Documentation ✅

- User guide for dashboard usage completed
- Developer documentation for extension added
- Real-world usage scenarios documented

Conclusion

Problem Restatement

- Kernel development often faces unreachable systems.

VGADash Solution

- Kernel-resident dashboard provides real-time system state via VGA text mode.

Technical Reliability

- VGA text mode offers a fundamental, stable interface.

Practical Debugging Value

- Enables immediate diagnosis of boot failures and system hangs.

Academic Contribution

- Demonstrates practical kernel subsystem integration.

VGADash provides critical observability in kernel development, enabling faster debugging and more robust code.